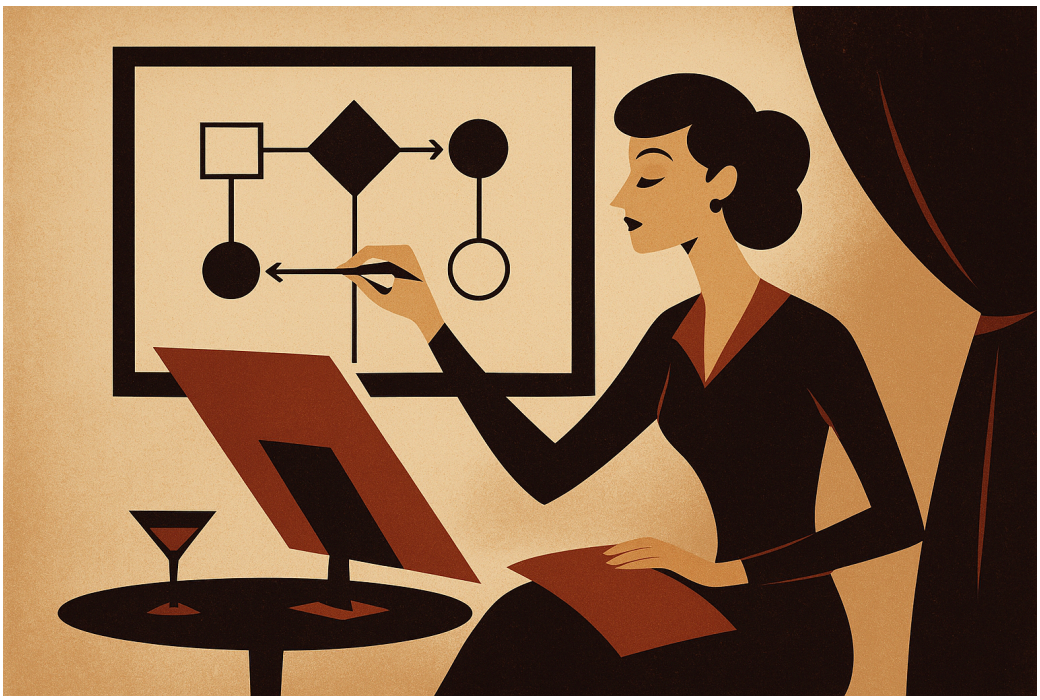
Assignment 4: Next λ-Calculus Interpreter (let*, let rec, Store, if/else)

Nitheesh Chandra Y
nitheeshchandra.y@research.iiit.ac.in
SERC, IIIT-H

PoPL | October 2025

Abstract



This assignment extends a basic λ-calculus interpreter to include sequential bindings (`let*`), mutual recursion (`let rec`), mutable state (references with `ref`, `deref`, and `set`), conditionals (`if/else`), booleans, sequencing, and primitive operations. Students will implement an operational semantics that threads a store through evaluations, allowing for stateful computations. The assignment also encourages exploration of different design choices and their implications on language semantics and implementation.

Keywords λ-Calculus · Interpreter · Operational Semantics · Let* · Let Rec · Store · References · Conditionals · Booleans · Sequencing · Primitives

1 Overview

> **Important**
>
> - You can implement this assignment in any programming language of your choice.
> - You can come up with your own names for the constructs if you want.
> - Parser is optional (extra credit); you can directly construct the AST in your language.
> - You can use any data structures you like to represent the AST and environment/store.
> - debug constructs like `print` are optional.
> - I am available for any clarifications. Please reach out via email or dm.
> - Appendix contains the solution to the previous assignment in Lean with code at GitHub Gist.

This **Next Assignment** extends your previous interpreter. You must:

- Add `let*` (sequential bindings),
- Add `let rec` for mutual recursion (both environment-only and store-based strategies),
- Add a mutable store with `ref`, `deref`, `set`,
- Add `if/else`, booleans, sequencing `e1; e2`,
- Update evaluation judgment to thread the store: `Γ; Σ ⊢ e => v ; Σ'`
  - Starting from an env `Γ` and store `Σ`
  - evaluating `e` yields value `v` and updated store `Σ'`.
- Implement primitives: arithmetic, comparisons, boolean ops, `print` (optional).

2 Specification

Let the sets of **expressible** and **denotable** values be:

$$\text{expressible Value} ::= n \mid \text{true} \mid \text{false} \tag{1}$$

$$\text{denoteable Value} ::= \text{Num} \mid \text{Bool} \mid \text{Clo} \mid \text{Loc} \tag{2}$$

$$\text{Clo} = \langle \text{literals id}^*, \text{body Exp}, \text{env Env} \rangle \tag{3}$$

$$\Gamma : \text{Env} = \text{id} \xrightarrow{\text{fin}} \text{Denotable Value} \tag{4}$$

$$\Sigma : \text{Store} = \text{loc} \xrightarrow{\text{fin}} \text{Value} \tag{5}$$

3 Syntax

$$\begin{aligned} e ::= {} & n \mid \text{true} \mid \text{false} \mid x \mid \lambda x.e \mid @ee...e \\ & \mid \text{let } ([x\ e]...)\ e \mid \text{letrec } ([x\ e]...)\ e \\ & \mid \text{ref } e \mid \text{deref } e \mid \text{set } x\ e \mid \\ & \mid \text{if } e\ e\ e \mid e\ ;\ e \end{aligned} \tag{6}$$

Primitives: `+ - * / == < <= > >= and or not print` (implement as needed).

4 Semantic Domains

Extend previous domains to include booleans and locations.

$$v ::= n \mid \text{true} \mid \text{false} \mid \langle x, e, \Gamma \rangle \mid \text{loc } \ell \tag{7}$$

Environments and store:

$$\begin{aligned} \Gamma : \mathrm{id} &\rightharpoonup d \\ \Sigma : \ell &\rightharpoonup v \end{aligned} \tag{8}$$

Evaluation judgment:

$$\begin{aligned} \Gamma \vdash e &\Rightarrow v \\ \Sigma \vdash \ell &\rightharpoonup v \\ \Gamma; \Sigma \vdash e &\Rightarrow v; \Sigma' \end{aligned} \tag{9}$$

- Γ maps variables to denotable values d (numbers, booleans, closures, locations).
- Σ maps locations ℓ to values v.
- ref e allocates a fresh location ℓ in Σ storing the value of e and returns ℓ.
- deref e retrieves the value at location ℓ.
- set e_1 e_2 updates the location ℓ from e_1 to hold the value of e_2.

> **Exploding Enviroments?**
>
> - Typically, environments grow with `let*` and `let rec`, while stores grow with `ref`.
> - And these persist even if the rest of the program dosen't need them anymore.
> - We could have reference counting or garbage collection, but that's beyond this assignment's scope.

5 Implementation notes & choices

- Thread a `Σ` (store) through your evaluator. Use either immutable maps (returned each call) or a mutable store object.
- Represent `loc` ℓ as unique integers or objects.
- `set` may return `unit`(nothing) or the assigned value or the location; be consistent in tests and report your choice.

6 Operational Semantics (rules)

> **Note**
>
> - You can also choose to do if-else is a trinary operator primitive.
> - Minimality MUST be ignored for these rules; as our input is AST.
>   - So no acceptance of "`let*` is just syntactic sugar!"

Below are the main inference rules.

$$\text{NUM}\ \frac{}{\Gamma; \Sigma \vdash n \Rightarrow \bar{n}; \Sigma}$$

$$\text{BOOL}\ \frac{}{\Gamma; \Sigma \vdash \text{true} \Rightarrow \text{true}; \Sigma \mid \Gamma; \Sigma \vdash \text{false} \Rightarrow \text{false}; \Sigma}$$

$$\text{VAR}\ \frac{v = \text{lookup}_{\text{env}}(\Gamma, x)}{\Gamma; \Sigma \vdash x \Rightarrow v; \Sigma}\ \text{lookup}_{\text{env}} : \Gamma \times x \to v$$

$$\text{ABS}\ \frac{}{\Gamma; \Sigma \vdash (\lambda x.e) \Rightarrow \langle x, e, \Gamma \rangle; \Sigma}$$

$$\text{APP-CLO}\ \frac{\Gamma; \Sigma \vdash e_0 \Rightarrow \langle x, e, \Gamma' \rangle; \Sigma_0 \quad \Gamma; \Sigma_0 \vdash e_1 \Rightarrow v_1; \Sigma_1 \quad \alpha = \{x \mapsto v_1\} \quad \alpha\Gamma'; \Sigma_1 \vdash e \Rightarrow v; \Sigma'}{\Gamma; \Sigma \vdash @e_0 e_1 \Rightarrow v; \Sigma'}$$

$$\text{APP-PRIM}\ \frac{\Gamma; \Sigma \vdash e_0 \Rightarrow \langle \text{prim\_literals}, p, \bot \rangle; \Sigma_0 \quad \Gamma; \Sigma_0 \vdash e_i \Rightarrow v_i; \Sigma_i \quad p(v_1, ..., v_n) = v}{\Gamma; \Sigma \vdash @e_0 e_1 ... e_n \Rightarrow v; \Sigma_n}$$

$$\text{REF}\ \frac{\Gamma; \Sigma \vdash e \Rightarrow v; \Sigma_0 \quad \ell = \text{create}(\Sigma_0) \quad \Sigma_1 = \Sigma_0[\ell \mapsto v]}{\Gamma; \Sigma \vdash \text{ref } e \Rightarrow \text{loc } \ell; \Sigma_1}\ \text{create} : \Sigma \to \text{loc } \ell$$

$$\text{DEREF}\ \frac{\Gamma; \Sigma \vdash e \Rightarrow \text{loc } \ell; \Sigma_0 \quad v = \text{lookup}_{\text{store}}(\Sigma_0, \ell)}{\Gamma; \Sigma \vdash \text{deref } e \Rightarrow v; \Sigma_0}\ \text{lookup}_{\text{store}} : \Sigma \times \ell \to v$$

$$\text{SET}\ \frac{\Gamma; \Sigma \vdash e_1 \Rightarrow \text{loc } \ell; \Sigma_1 \quad \Gamma; \Sigma_1 \vdash e_2 \Rightarrow v; \Sigma_2 \quad \Sigma_3 = \Sigma_2[\ell \mapsto v]}{\Gamma; \Sigma \vdash \text{set } e_1 e_2 \Rightarrow v; \Sigma_3}$$

$$\text{IF-TRUE}\ \frac{\Gamma; \Sigma \vdash e_1 \Rightarrow \text{true}; \Sigma_1 \quad \Gamma; \Sigma_1 \vdash e_2 \Rightarrow v; \Sigma_2}{\Gamma; \Sigma \vdash \text{if } e_1 e_2 e_3 \Rightarrow v; \Sigma_2}$$

$$\text{IF-FALSE}\ \frac{\Gamma; \Sigma \vdash e_1 \Rightarrow \text{false}; \Sigma_1 \quad \Gamma; \Sigma_1 \vdash e_3 \Rightarrow v; \Sigma_2}{\Gamma; \Sigma \vdash \text{if } e_1 e_2 e_3 \Rightarrow v; \Sigma_2}$$

$$\text{SEQ}\ \frac{\Gamma; \Sigma \vdash e_1 \Rightarrow v_1; \Sigma_1 \quad \Gamma; \Sigma_1 \vdash e_2 \Rightarrow v_2; \Sigma_2}{\Gamma; \Sigma \vdash e_1; e_2 \Rightarrow v_2; \Sigma_2}$$

$$\text{LET*}\ \frac{\Gamma; \Sigma \vdash e_1 \Rightarrow v_1; \Sigma_1 \quad \Gamma_1 = \Gamma[x_1 \mapsto v_1] \quad \Gamma_1; \Sigma_1 \vdash e_2 \Rightarrow v_2; \Sigma_2 \quad \Gamma_2 = \Gamma_1[x_2 \mapsto v_2] \quad ... \quad \Gamma_k; \Sigma_k \vdash e_{\text{body}} \Rightarrow v; \Sigma'}{\Gamma; \Sigma \vdash \text{let*}([x_1 e_1]...[x_k e_k]) e_{\text{body}} \Rightarrow v; \Sigma'}$$

$$\text{LET-REC}\ \frac{\Gamma_r = \Gamma[\forall i \mid f_i \mapsto \langle f_i, e_i, \bot \rangle] \quad \Sigma_0 = \Sigma \quad \Gamma_r; \Sigma_{i-1} \vdash e_i \Rightarrow v_i; \Sigma_i \quad \Gamma_{r'} = \Gamma_r[f_i \mapsto v_i] \quad \Gamma_{r'}; \Sigma_k \vdash \text{body} \Rightarrow v; \Sigma'}{\Gamma; \Sigma \vdash \text{let rec } ([f_1 e_1]...) \text{ body} \Rightarrow v; \Sigma'}$$

Table 1: Evaluation Rules

7 Suggested primitives (up to you)

Implement these primitive functions (as $\langle \text{prim\_literals}, \text{p\_body}, \bot \rangle$ values in the initial environment):

- Arithmetic: `+ - * /` : numbers → number (error on type mismatch; `/` may be integer or rational).
- Comparisons: `== < <= > >=` : numbers → booleans.
- Boolean ops: `and` (short-circuit), `or` (short-circuit), `not`.
- `print` : any → unit (side-effect; useful for debugging).
- `unit` literal.

Document each primitive in your README/report: arity, argument types, return type, and behavior on bad types.

8 Additional Explorations

8.1 Alternative `let*` Rule

Consider the following variations:

$$\text{LET}_1\text{*}\ \frac{\Gamma;\Sigma \vdash e_1 \Rightarrow v_1;\Sigma_1 \quad \Gamma_1 = \Gamma[x_1 \mapsto v_1] \quad \Gamma_1;\Sigma_1 \vdash e_2 \Rightarrow v_2;\Sigma_2 \quad \Gamma_2 = \Gamma_1[x_2 \mapsto v_2] \quad \cdot \quad \Gamma_k;\Sigma_k \vdash e_{\text{body}} \Rightarrow v;\Sigma'}{\Gamma;\Sigma \vdash \text{let*}([x_1 e_1]...[x_k e_k])e_{\text{body}} \Rightarrow v;\Sigma'}$$

$$\text{LET}_2\text{*}\ \frac{\Gamma;\Sigma \vdash e_1 \Rightarrow v_1;\Sigma_1 \quad \Gamma_1 = \Gamma[x_1 \mapsto v_1] \quad \Gamma;\Sigma_1 \vdash e_2 \Rightarrow v_2;\Sigma_2 \quad \Gamma_2 = \Gamma_1[x_2 \mapsto v_2] \quad ... \quad \Gamma_k;\Sigma_k \vdash e_{\text{body}} \Rightarrow v;\Sigma'}{\Gamma;\Sigma \vdash \text{let*}([x_1 e_1]...[x_k e_k])e_{\text{body}} \Rightarrow v;\Sigma'}$$

Table 2: Alternative `let*` Rule

- Reason about the difference between the two `let*` rules above.
- Implement both `let*` rules above and compare behavior.
- Can you come up with an example that behaves differently under the two rules?

8.2 No `ref` and `deref`

- Implement `set!` to take a variable name instead of a location: `(set! x e)` updates the binding of `x` in the environment to the value of `e` in the current store.
- Adjust and mention the rules accordingly.
- Discuss pros and cons of this design.
- Implement both `set!` designs and compare behavior.
- Can you come up with a behavior that is not possible in one design but possible in the other?

8.3 Minimalism (v2)

We know that

`let` can be encoded using lambda and application.

$$\text{let } [xe]e' \triangleq @(\lambda x.e')e \tag{10}$$

Can you do similar ones for:

> **Discuss in report**
>
> - `let*`
> - `let rec`
> - `if/else`
> - `ref`, `deref`, `set`
> - `;` (sequencing)

- Mention the **rules** and implementation changes needed to support these encodings.
- Discuss the **trade-offs** of using the core language vs adding these constructs directly.

9 Report component (what to submit)

Provide a short **1–2 page** report including:

1. An **annotated AST** of one non-trivial example (e.g., `let rec` factorial or `even/odd`) showing closures `⟨x, e, Γ⟩` and any `loc` ℓ allocations. i.e Store Σ snapshots.
2. The **list of primitives** you implemented with brief semantics. (You can also build your own primitives) arity, argument types, return type, and behavior on bad types.
3. A **summary of your design choices** (e.g., mutable vs immutable store, `let rec` strategy).
4. Any **challenges** you faced and how you overcame them.
5. Any **extra credit** features you implemented (e.g., error handling, additional primitives, optimizations). (*TA discretion*)
6. A README describing how to run the interpreter and the test suite.

10 Bonus (extra credit)

- Parser implementation (from string to AST).
- Replace `Num` with `Array` (arrays) and its primitives.
- Any other interesting feature you can think of (discuss with me first)
- As most editors and languages support Unicode, What if you use some symbols to represent constructs? Checkout https://www.uiua.org/

Probabilistic Programming Languages

- Consider flipping a coin (random number `{0, 1}`), could you create such a primitive `flip`? can you specify a optional probability distribution over the outcomes?

Following is not in a syntax this document follows, but just an idea.

```
var a = flip(0.3);
var b = flip(0.3);
var c = flip(0.3);
return a + b + c  // a random sample from Binomial distribution B(3, 0.3)
```

May be you can also have primitive `repeat(n, e)` that evaluates `e` `n` times and returns an array of results. And a `viz` to visualize the results.

```
var results = repeat(1000, flip(0.3));
viz(results); // histogram of results
```

What about other distributions like Normal, Poisson, Exponential, etc.?

- How would you conditional probability constructs like `observe` or `condition` or `bayes`?
- What abot non-deterministic constructs like `choose` or `amb`?

Exploring Other Programming Paradigms

- Explore logic programming (e.g., Prolog) to represent constructs like `let rec` as mutually recursive predicates and `if/else` as guarded rules or alternative facts.
- Investigate functional reactive programming (e.g., Elm) to model stateful constructs like `ref` and `set` as signals or subscriptions, enabling automatic propagation of state changes.
- Examine concatenative programming (e.g., Forth) to implement sequencing and state management using stack-based operations, where sequencing is implicit and state is manipulated via stack commands.
- Compare these paradigms with λ-calculus-based languages in terms of expressiveness, simplicity, and implementation complexity, highlighting trade-offs like minimalism vs practical concurrency or declarative reasoning.

APPENDIX A Example Tests

> **Warning**
>
> In case of any issues, please send me the corrections

> **Important**
>
> Due to the choice of your language and the corresponding primitives might differ. Please use if your judgement to modify the tests accordingly.
>
> If ignored, please mention the reasoning in your report. (*TA discretion*)

1. Numeric literal

```
42
```

→ `42`

2. `let*` sequential binding

```
let* ([x 1]
      [y 2])
@ + x y
```

→ 3

3. `let*` shadowing (optional and precedence can be discussed in report)

```
let* ([x 1]
       [x 6])
x
```

→ `6` or `1` or error (discuss in report)

4. `let rec` mutual (even/odd)

```
let rec ([even (λ n. if (@ == n 0) true (@ odd (@ - n 1)))]
         [odd  (λ n. if (@ == n 0) false (@ even (@ - n 1)))])
(@ even 4)
```

→ `true`

5. `let rec` factorial

```
let rec ([fact (λ n.
 if (@ == n 0) 1
 (@ * n (@ fact (@ - n 1))))])
(@ fact 5)
```

→ `120`

6. Store allocation & mutation

```
let ([r (ref 10)])
(set r (@ + (deref r) 5))
(deref r)
```

→ `15`

7. Closures capturing locations

```
let ([r (ref 10)]
     [adder (λ x. set r (@ + (deref r) x))])
(@ adder 5);
(deref r)
```

→ 15

8. Sequencing

```
(set (ref 0) 1); 42
```

(Depending on your `set!` design this may be an error; prefer using a variable.)

→ 42

9. **Mutual recursion with higher-order arguments**

```
let rec ([f (λ g. (@ g 0))]
         [g (λ n. if (@ == n 0) 1 (@ f g))])
(@ f g)
```

10. **Store + recursion**

```
let ([counter (ref (λ n. (* 2 n)))])
  (let ([old (deref counter)])                ; capture previous function
    (set counter (λ n. (@ + ((old) n) 1)));  ; new function calls old, not itself
    (@ (deref counter) 2))
```

11. **Iterator using store**

```
let ([pos (ref 0)]
      [it  (λ _. (let ([current (deref pos)])
                   (set pos (@ + current 1))
                   current))])
  ; call it three times
  (@ it ());
  (@ it ());
  (@ it ())
```

`0`, `1`, `2` on successive calls
→ `2` (last call result)

APPENDIX B Prev. Assignment solution in Lean

> **Code Also on Github Gist**
>
> The code below is also available as a GitHub Gist.
>
> Add comments on the Gist if you find any issues or have suggestions.

B.1 Interpreter

```
import Lean

open Lean Meta Widget
open Std

/-- Possible runtime errors. -/
inductive Error
  | varNotFound (x : String)
  | notAFunction (e : String)
  | badPrimitive (e : String)
  | operationOnWrongType (e t : String)
  | divByZero
deriving Repr

instance : ToString Error where
  toString
    | Error.varNotFound x         => s!"variable {x} not found"
    | Error.notAFunction e      => s!"{e} is not a function"
    | Error.badPrimitive e      => s!"{e} is not a valid primitive"
    | Error.operationOnWrongType e t => s!"operation {e} on wrong type {t}"
    | Error.divByZero             => "division by zero"

inductive PrimOp
  | add
  | sub
  | mul
  | div
  | eq
deriving Repr, BEq

instance : ToString PrimOp where
  toString
    | .add => "+"
    | .sub => "-"
    | .mul => "*"
    | .div => "/"
    | .eq  => "=="


/-- Syntax for λ-calculus expressions. -/
inductive LambdaCalc where
  | const (n : Int)
  | var (x : String)
  | lam (x : String) (body : LambdaCalc)
  | app (e₁ e₂ : LambdaCalc)
```

```
  | appPrim (prim : PrimOp) (e₁ e₂ : LambdaCalc)
  | letIn (x : String) (e₁ e₂ : LambdaCalc)
deriving Repr

def lambdaCalcToString : LambdaCalc → String
  | .const n => toString n
  | .var x => x
  | .lam x e => s!"(λ {x}. {lambdaCalcToString e})"
  | .app e₁ e₂ => s!"(@ {lambdaCalcToString e₁} {lambdaCalcToString e₂})"
  | .appPrim prim e₁ e₂ => s!"({prim} {lambdaCalcToString e₁} {lambdaCalcToString
e₂})"
  | .letIn x e₁ e₂ => s!"(let [{x} {lambdaCalcToString e₁}] {lambdaCalcToString e₂})"

instance : ToString LambdaCalc where
  toString := lambdaCalcToString

/- change repr to show full AST Tree but not as a String -/
instance : Repr LambdaCalc where
  reprPrec e _ := toString e


/- Value domain: numbers, closures, and primitives. -/
inductive Value : Type where
  | num  (n : Int)
  | clo  (x : String) (body : LambdaCalc) (env : List (String × Value))
  | prim (p : PrimOp)
deriving Repr, Nonempty

instance : ToString Value where
  toString
    | .num n => toString n
    | .clo x _ _ => s!"<closure {x}>"
    | .prim p => s!"<prim {p}>"


/-- Environment is a list of variable bindings.
Alternately, we could use other data structures;
-/
abbrev Env := List (String × Value)

/-- Lookup in environment. -/
def lookup (env : Env) (x : String) : Except Error Value :=
  match env.find? (fun p => p.fst = x) with
  | some (_, v) => Except.ok v
  | none =>
    match x with
    | "+" => Except.ok (Value.prim .add)
    | "-" => Except.ok (Value.prim .sub)
    | "*" => Except.ok (Value.prim .mul)
    | "/" => Except.ok (Value.prim .div)
    | "==" => Except.ok (Value.prim .eq)
    | _     => Except.error (Error.varNotFound x)

/-- Apply a primitive to Int arguments.
    Here: +, -, *, /, == are primitives from Lean.
```

```
 -/
def applyPrim (p : PrimOp) (args : List Value) : Except Error Value := do
  let ns ← args.mapM fun
    | .num n => pure n
    | _ => Except.error (.badPrimitive s!"Non-numeric argument for {p}")
  match p, ns with
  | .add, [a,b] => pure (.num (a + b))
  | .sub, [a,b] => pure (.num (a - b))
  | .mul, [a,b] => pure (.num (a * b))
  | .div, [a,b] => if b == 0 then Except.error .divByZero else pure (.num (a / b))
  | .eq,  [a,b] => pure (.num (if a == b then 1 else 0))
  | _, _ => Except.error (.badPrimitive s!"wrong arity for {p}")


partial def eval (env : Env) : LambdaCalc → Except Error Value
  | .const n => pure (.num n)
  | .var x   => lookup env x
  | .lam x e => pure (.clo x e env)
  | .app e₁ e₂ => do
      let v₁ ← eval env e₁
      let v₂ ← eval env e₂
      match v₁ with
      | .clo x body cloEnv =>
        eval ((x,v₂)::cloEnv) body
      | .prim p => Except.error (.notAFunction (s!"primitive {p}"))
      | _ => Except.error (.notAFunction (toString v₁))
  | .appPrim prim e₁ e₂ => do
      let v₁ ← eval env e₁
      let v₂ ← eval env e₂
      applyPrim prim [v₁, v₂]
  | .letIn x e₁ e₂ => do
      let v₁ ← eval env e₁
      eval ((x,v₁)::env) e₂

/-- Pretty printer for result. -/
def evalPP (e : LambdaCalc) : String :=
  match eval [] e with
  | .ok v => s!"Result: {v}"
  | .error err => s!"Error: {err}"


declare_syntax_cat LambdaCalcSyntax
syntax num : LambdaCalcSyntax
syntax ident : LambdaCalcSyntax
syntax str : LambdaCalcSyntax
syntax "(" LambdaCalcSyntax ")" : LambdaCalcSyntax
syntax "λ" ident "." LambdaCalcSyntax : LambdaCalcSyntax
syntax "@" LambdaCalcSyntax LambdaCalcSyntax : LambdaCalcSyntax
syntax:65 LambdaCalcSyntax:65 "+" LambdaCalcSyntax:66 : LambdaCalcSyntax
syntax:65 LambdaCalcSyntax:65 "-" LambdaCalcSyntax:66 : LambdaCalcSyntax
syntax:70 LambdaCalcSyntax:70 "*" LambdaCalcSyntax:71 : LambdaCalcSyntax
syntax:70 LambdaCalcSyntax:70 "/" LambdaCalcSyntax:71 : LambdaCalcSyntax
syntax:50 LambdaCalcSyntax:50 "==" LambdaCalcSyntax:51 : LambdaCalcSyntax
syntax "let" "[" ident LambdaCalcSyntax "]" LambdaCalcSyntax : LambdaCalcSyntax
```

```
syntax "`[LC| " LambdaCalcSyntax "]" : term

macro_rules
  | `(`[LC| $n:num]) => `(LambdaCalc.const $n)
  | `(`[LC| $x:ident]) => `(LambdaCalc.var $(Lean.Syntax.mkStrLit x.getId.toString))
  | `(`[LC| ( $e:LambdaCalcSyntax )]) => `(`[LC| $e])
  | `(`[LC| λ $x:ident . $body:LambdaCalcSyntax]) => `(LambdaCalc.lam
$(Lean.Syntax.mkStrLit x.getId.toString) `[LC| $body])
  | `(`[LC| @ $e₁:LambdaCalcSyntax $e₂:LambdaCalcSyntax]) => `(LambdaCalc.app `[LC|
$e₁] `[LC| $e₂])
  | `(`[LC| $e₁:LambdaCalcSyntax + $e₂:LambdaCalcSyntax]) =>
`(LambdaCalc.appPrim .add `[LC| $e₁] `[LC| $e₂])
  | `(`[LC| $e₁:LambdaCalcSyntax - $e₂:LambdaCalcSyntax]) =>
`(LambdaCalc.appPrim .sub `[LC| $e₁] `[LC| $e₂])
  | `(`[LC| $e₁:LambdaCalcSyntax * $e₂:LambdaCalcSyntax]) =>
`(LambdaCalc.appPrim .mul `[LC| $e₁] `[LC| $e₂])
  | `(`[LC| $e₁:LambdaCalcSyntax / $e₂:LambdaCalcSyntax]) =>
`(LambdaCalc.appPrim .div `[LC| $e₁] `[LC| $e₂])
  | `(`[LC| $e₁:LambdaCalcSyntax == $e₂:LambdaCalcSyntax]) =>
`(LambdaCalc.appPrim .eq `[LC| $e₁] `[LC| $e₂])
  | `(`[LC| let [ $x:ident $e₁:LambdaCalcSyntax ] $e₂:LambdaCalcSyntax]) =>
      `(LambdaCalc.letIn $(Lean.Syntax.mkStrLit x.getId.toString) `[LC| $e₁] `[LC|
$e₂])
      -- Above let can be changed to use λ and @ only, if desired.
      -- `(LambdaCalc.app (LambdaCalc.lam $(Lean.Syntax.mkStrLit x.getId.toString)
`[LC| $e₂]) `[LC| $e₁])


/-- Example expressions matching assignment specification. -/
def ex1 := `[LC| 42]
def ex2 := `[LC| x]
def ex3 := `[LC| 1 + 2 * 3 - 10 / 5]
def ex4 := `[LC| λ x. x + 1]
def ex5 := `[LC| @ (λ x. x + 1) 5]
def ex6 := `[LC| let [ x 5 ] x + 3]
def ex7 := `[LC| @ (@ (λ x. λ y. x + y) 3) 4]

#eval evalPP ex1
#eval evalPP ex2
#eval eval [("x", .num 10)] ex2
#eval evalPP ex3
#eval evalPP ex5
#eval evalPP ex6
#eval evalPP ex7


#eval toString ex1
#eval toString ex2
#eval toString ex3
#eval toString ex4
#eval toString ex5
#eval toString ex6
#eval toString ex7
```